**Spike:** 1
**Title:** Graphs, Search and Rules
**Author:** Pattarapol Tantechasa, 103883220

**Goals:**

1. Tic Tac Toe code modified to represent the game state as a graph.
2. An AI that randomly searches the graph.
3. An AI that improves the efficiency of a basic random search.
4. An AI that improves the effectiveness of a basic random search.

**Deliverables:**

- Spike Plan (BitBucket repo as md file)
- Python Code (BitBucket)
- Spike Outcome Report

**Technologies, Tools, and Resources used:**

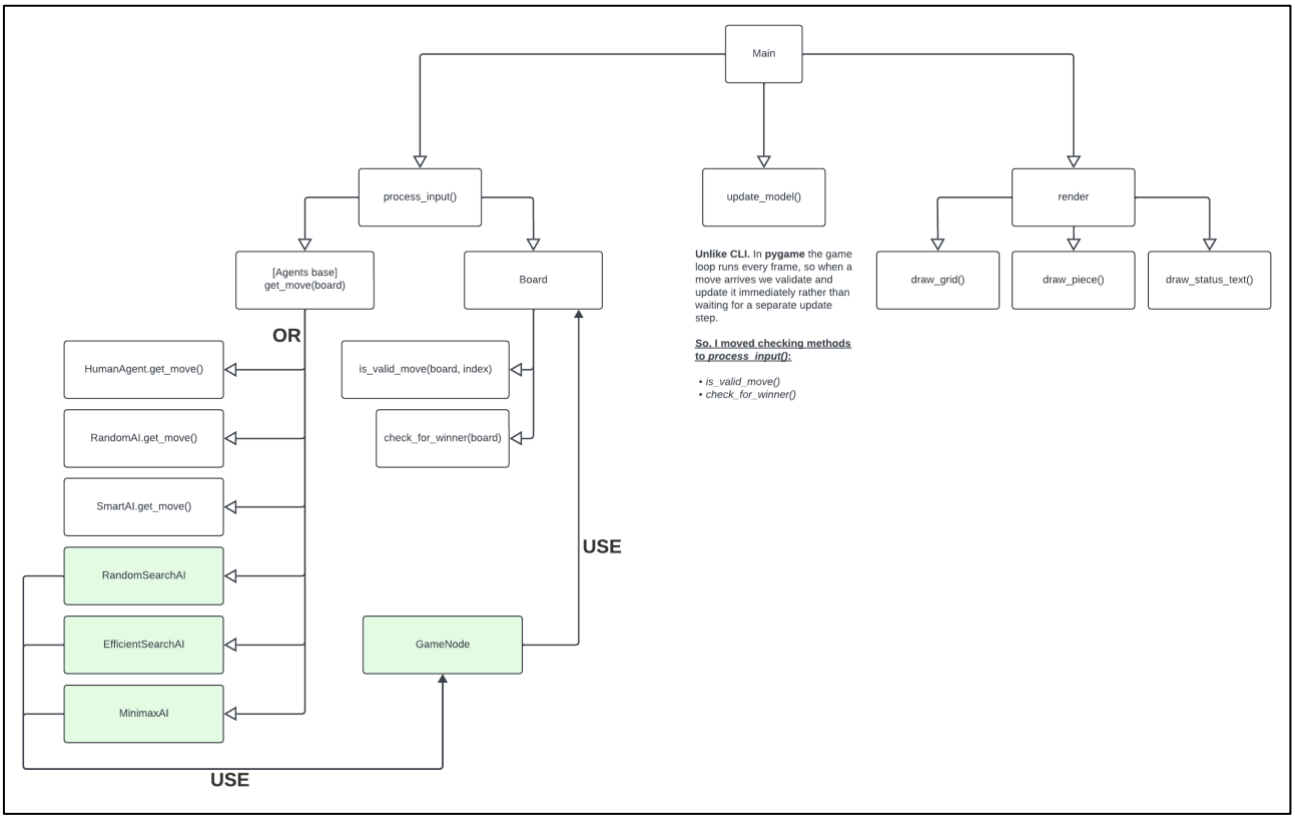
- Minimax Algorithm for Tic Tac Toe
- Claude: asking BFS & DFS concept relate question and brainstorm ideas of how to integrate to the game
- Visual Studio Code IDE
- Conda: Python environment manager

**Tasks undertaken:**

1. Download and Install **Visual Studio Code**
2. Install **Python** on your Machine
3. Install **PyGame**
4. Open the main project downloaded from
5. Open terminal and run cd command : **cd Assignments-Labs/Assignment-2/Task-4-Spike-Graph-Search/**
6. Run: **python main.py**
    a. this will run the game with the chosen player
    b. you can look in **main.py** to choices or modify player yourself

**What we found out:**

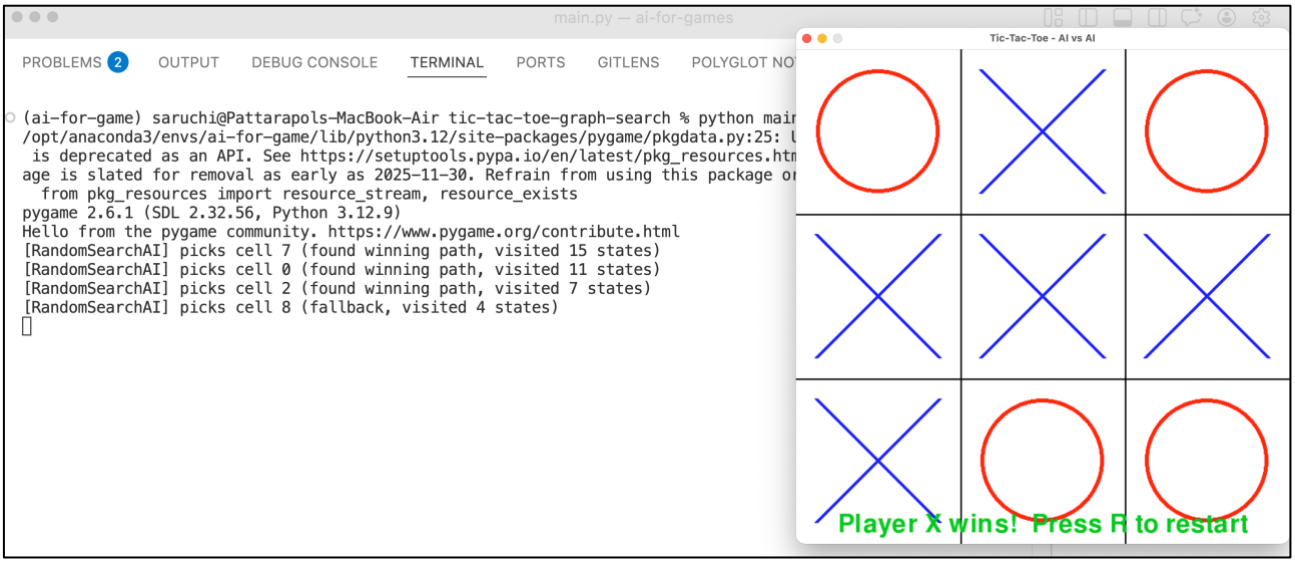
The **Tic Tac Toe** board can be modelled as a graph where each board state is a node, and each valid move is a directed edge to a child state. The **GameNode** class captures this, it stores a snapshot of the board, the move that produced it, and a **parent** pointer so the AI can trace a path from a future winning state back to the move it should make right now.




## Outcome 1: Random graph search (RandomSearchAI)

The AI uses **DFS with a stack** to randomly walk the game state graph. When a winning state is found, it walks the **parent** chain back to the first move from the real board and returns it. **DFS** is a natural fit because the goal is to reach a terminal state (win/lose/tie) as fast as possible

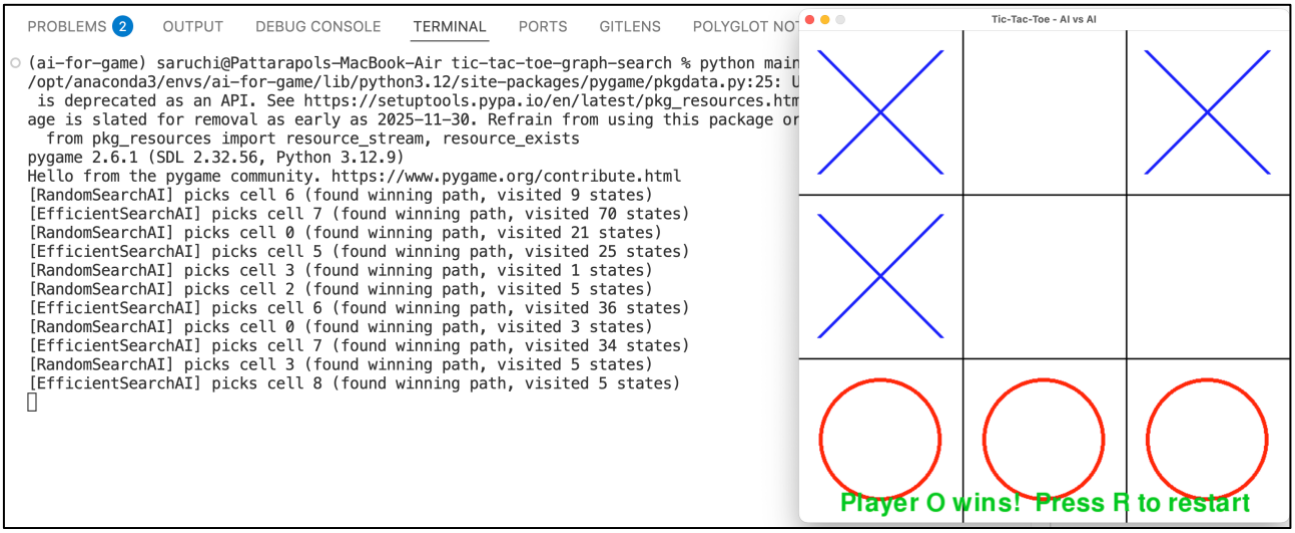
However, this agent is both **inefficient** (may revisit the same board states via different move orderings) and **ineffective** (simulates the opponent randomly, so "winning paths" it finds may not work against a real opponent).




## Outcome 2: Efficient graph search (EfficientSearchAI)

Adds a **visited** set to the **DFS** walk. Each board state is converted to a **tuple** (hashable) and stored in the set before pushing a new node onto the stack, it checks if that state has already been explored. This avoids re-processing duplicate board positions that can be reached via different move orderings.

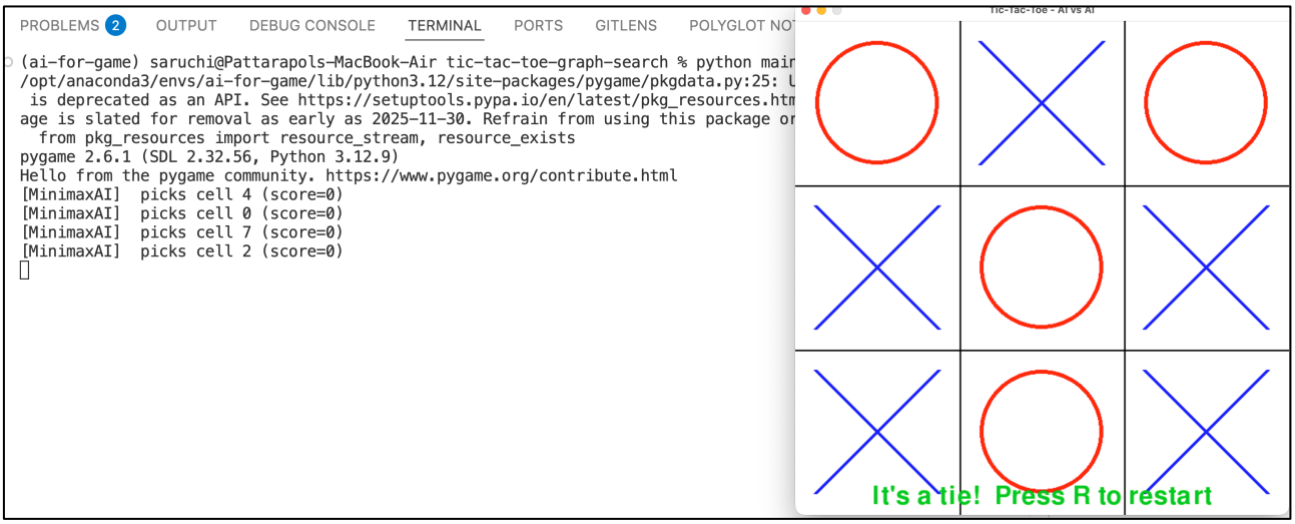
In practice the improvement in visited state count is modest because the random walk only expands one child per node, so few duplicates are generated on a single path. Deduplication becomes more impactful when the search fans out broadly.

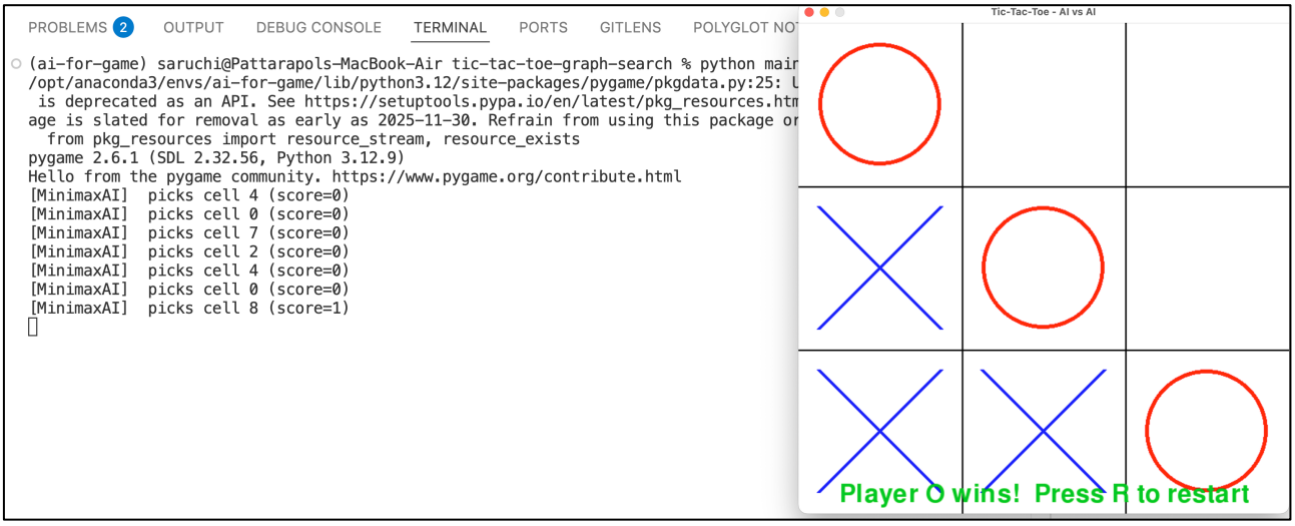



## Outcome 3: Minimax (MinimaxAI)

The most effective approach. Instead of a random walk, **MinimaxAI** exhaustively searches the entire game tree using recursion. Every leaf node

is scored: **+1** if the AI wins, **-1** if the opponent wins, **0** for a tie. On the AI's turn it picks the move with the **highest** score. While on the opponent's turn it assumes they pick the move with the **lowest** score. This means **MinimaxAI** always assumes perfect play from both sides and never loses.



## Extension: Search Tree Visualisation

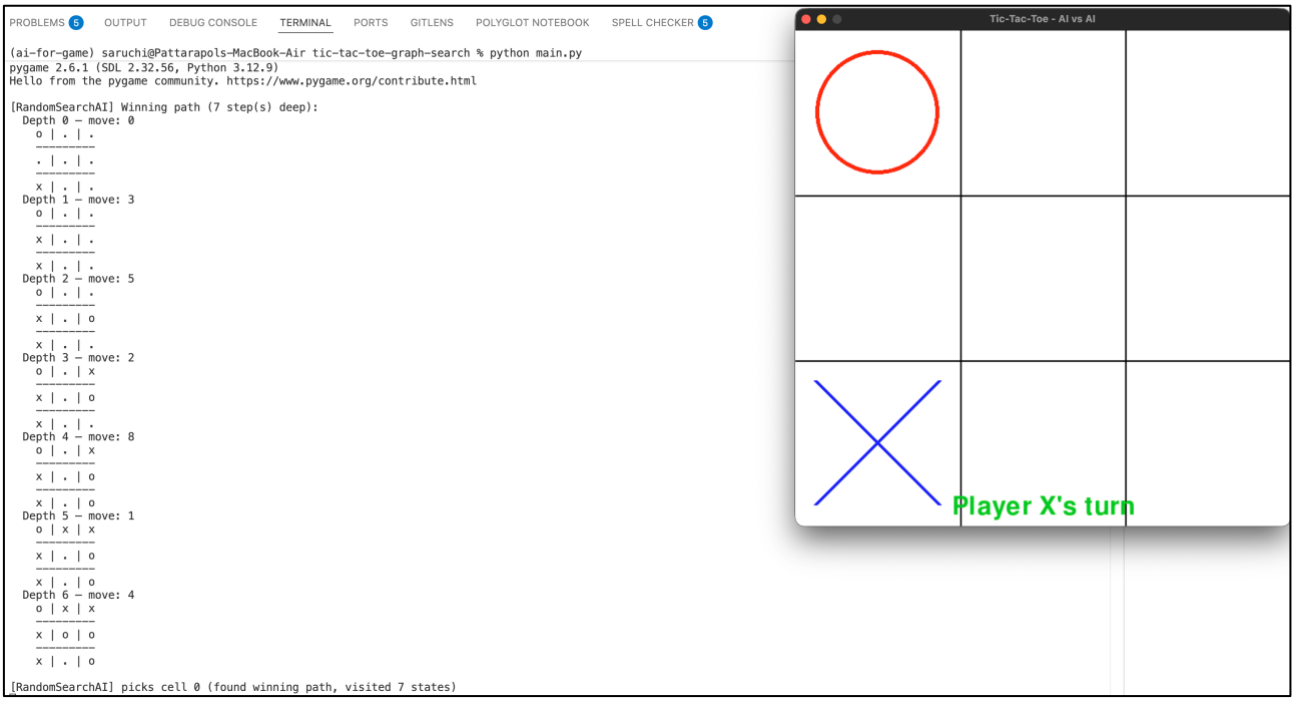
To make the AI's decision-making visible, a path visualiser was added to **RandomSearchAI** and **EfficientSearchAI**. When either agent finds a winning path, it prints the full sequence of board states from the first simulated move to the winning state directly to the console.

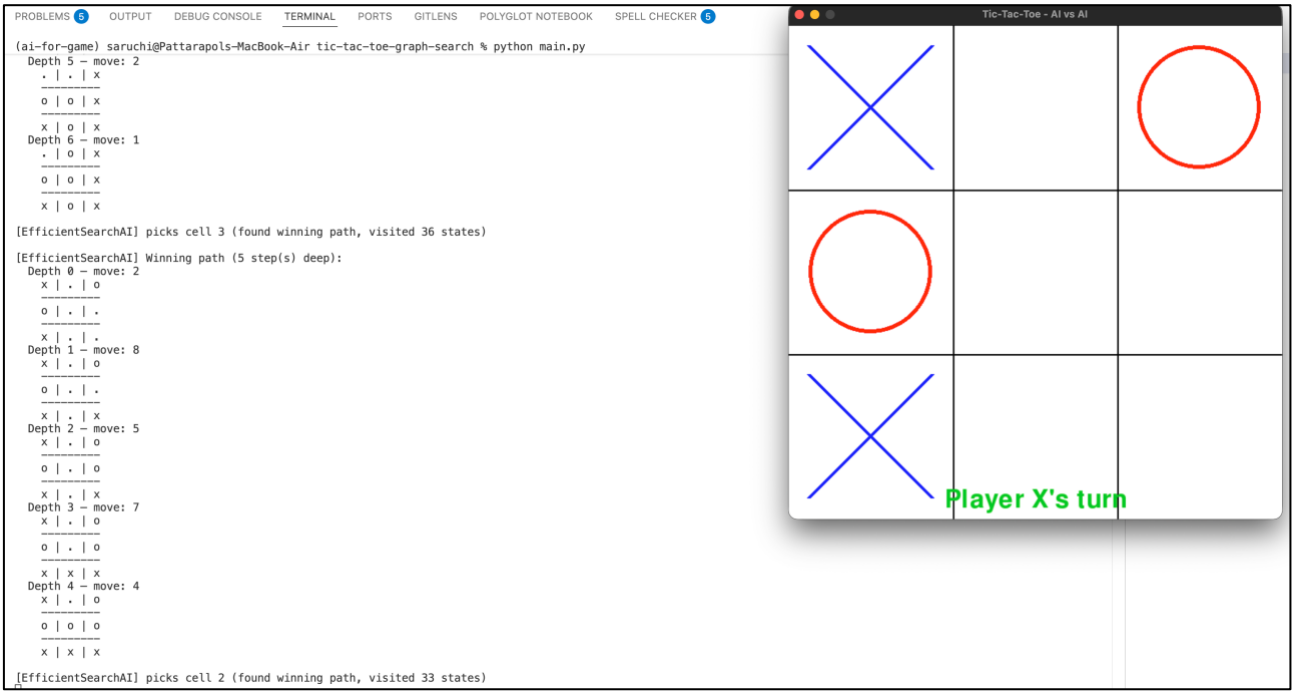
Two utility functions were added to **GameNode**:

- ***render_board(board_state)***: converts a board state list into a readable 3×3 ASCII grid, using '***. (dot)***' for empty
- ***print_path(winning_node, agent_name)***: walks the **GameNode** parent chain from the winning node back to the root, reverses the path, then prints each board state with its depth level and move index.

***print_path*** relies on the **GameNode** parent chain to reconstruct the path from root to winning state. **RandomSearchAI** and **EfficientSearchAI** build this chain explicitly each node stores a reference to the node that came before it, so the full path is traceable at the moment a win is found.

**MinimaxAI** works differently. It never commits to a single path during search. Instead, it scores every possible outcome by recursing through the entire game tree, then picks the move with the highest score at the root level. Because it evaluates all branches before deciding, there is no single "path taken" to trace. The winning move is identified by comparing scores across all candidates, not by following one chain of nodes to a terminal state. As a result, **MinimaxAI** has no parent chain to walk, making ***print_path*** not applicable.




**Open issues/risks:**

- **RandomSearchAI** and **EfficientSearchAI** do not simulate the opponent optimally. They may find a "winning path" that only works if the opponent plays poorly. In a real game this path may never occur.